

SPIN - Seminar

Simple On-the-fly Automatic Verification
of Linear Temporal Logic

Formal Methods for AI-Based Systems Engineering (FMAISE)

Robert Li & William Tossici (1983696, 1844504)

Academic Year 2025/2026



SAPIENZA
UNIVERSITÀ DI ROMA



Table of Contents

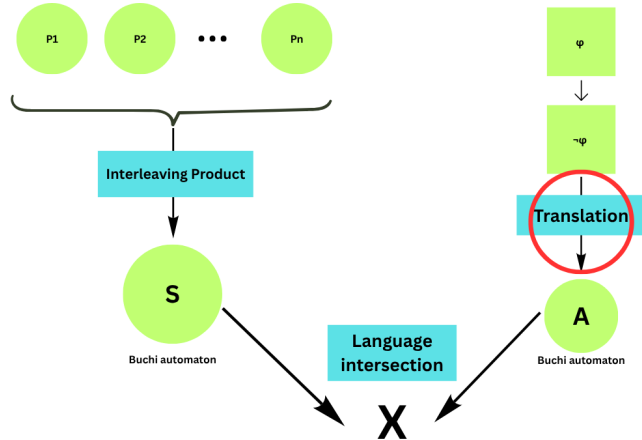
1 Abstract

- ▶ Abstract
- ▶ Introduction
- ▶ Preliminaries
- ▶ A Tableau Construction
 - The algorithm
 - Using the Automaton for Automatic Protocol Verification
- ▶ Example
- ▶ Nuances and summary
 - Improvements to the Basic Algorithm
- ▶ Experimental Results
- ▶ Conclusions
- ▶ Other stuff



What do we present today?

1 Abstract





Abstract: The Core Innovation

1 Abstract

A New Tableau-Based Algorithm

- **Goal:** Translate Linear Temporal Logic (LTL) formulas into automata.
- **Approach:** Strictly “on-the-fly” model checking.
- **Mechanism:** Automaton construction is guided by, and simultaneous with, the system model generation.

Key Practical Benefits

- **Early Bug Detection:** Violations are detected without building the entire automaton.
- **Dual Purpose:** Useful for both model checking and checking temporal logic validity.
- **High Efficiency:** Overcomes theoretical PSPACE-complete limits, performing excellently on typical verification formulas.



Table of Contents

2 Introduction

- ▶ Abstract
- ▶ **Introduction**
- ▶ Preliminaries
- ▶ A Tableau Construction
 - The algorithm
 - Using the Automaton for Automatic Protocol Verification
- ▶ Example
- ▶ Nuances and summary
 - Improvements to the Basic Algorithm
- ▶ Experimental Results
- ▶ Conclusions
- ▶ Other stuff



The Classical Approach

2 Introduction

How it historically worked:

1. Construct the state space for the **protocol**.
2. Construct the state space for the **negation** of the property.
3. Create the synchronous product of both automata.

The Emptiness Check

If the language accepted by the product automaton is *not* empty, a common execution sequence exists. This sequence represents a violation of the property, revealing a bug in the protocol.



The Complexity Wall: Memory Explosion

2 Introduction

Theoretical & Practical Limits

- The model-checking problem is mathematically PSPACE complete.
- **Time & Size:** The property automaton can have up to $2^{\mathcal{O}(n)}$ nodes, making the product size proportional to $N \cdot 2^{\mathcal{O}(n)}$.

The Memory Bottleneck (Global Construction)

Standard constructions generate the property automaton *globally* (all at once) before the check begins. This exponential blowup quickly exceeds available main memory, limiting the size of problems that can be treated.



The Solution: DFS and On-The-Fly Construction

2 Introduction

Depth-First Search (DFS) Advantage

- DFS cycle detection requires keeping only a small part of the automaton in memory at any one time (the current computation path).

The Paper's Innovation

- Translates LTL into a Generalized Büchi automaton using simple DFS.
- **True On-The-Fly:** Generates the property automaton *only as needed*.
- Both protocol and property automata are built simultaneously during the check, completely avoiding global exponential generation.



Table of Contents

3 Preliminaries

- ▶ Abstract
- ▶ Introduction
- ▶ **Preliminaries**
- ▶ A Tableau Construction
 - The algorithm
 - Using the Automaton for Automatic Protocol Verification
- ▶ Example
- ▶ Nuances and summary
 - Improvements to the Basic Algorithm
- ▶ Experimental Results
- ▶ Conclusions
- ▶ Other stuff



Preliminaries: LTL Syntax

3 Preliminaries

Building Blocks of Linear Temporal Logic

- Formulas are constructed from a finite set of **atomic propositions** $\mathcal{P}$.
- **Boolean Operators:** $\neg$ (NOT), $\wedge$ (AND), $\vee$ (OR).
- **Core Temporal Operators:** X (Next) and U (Until).

Inductive Definition

- Every member $q \in \mathcal{P}$ is a formula.
- If φ and ψ are formulas, then so are:
 $\neg\varphi, \quad \varphi \wedge \psi, \quad \varphi \vee \psi, \quad X\varphi, \quad \varphi U \psi.$



Interpretations: The Concept of “Time”

3 Preliminaries

How does the automaton read a program?

- An interpretation is an **infinite word** $\xi = x_0x_1x_2 \dots$
- The alphabet is $2^{\mathcal{P}}$ (all possible subsets of atomic propositions).

What does it mean?

Think of ξ as an infinite film strip of the protocol's execution. Each frame (x_i) represents a state of the system, determining exactly which propositions are *true* and which are *false* at that specific moment.



Semantics: When is a formula true?

3 Preliminaries

Let ξ_i be the suffix of the execution starting at frame x_i .

The Basic Rules ($\models$)

- $\xi \models q \iff q \in x_0$ (True right now).
- **Next (X):** $\xi \models X\varphi \iff \xi_1 \models \varphi$
Meaning: φ will be true in the very next state.

The “Until” Operator (U)

$\xi \models \varphi \text{ U } \psi \iff \exists i \geq 0$ such that $\xi_i \models \psi$ **and** $\xi_j \models \varphi$ for all $0 \leq j < i$.

*Meaning: φ must hold continuously **until** ψ eventually becomes true.*



Table of Contents

4 A Tableau Construction

- ▶ Abstract
- ▶ Introduction
- ▶ Preliminaries
- ▶ A Tableau Construction
 - The algorithm
 - Using the Automaton for Automatic Protocol Verification
- ▶ Example
- ▶ Nuances and summary
 - Improvements to the Basic Algorithm
- ▶ Experimental Results
- ▶ Conclusions
- ▶ Other stuff



A Tableau Construction: The Target Automaton

4 A Tableau Construction

The Main Goal

- Build a transition system generating all infinite sequences satisfying a given formula φ .
- **Choice of Automaton:** Generalized Büchi Automata (GBA).

Why Generalized?

Unlike simple Büchi automata (which have only one set of accepting states), a GBA has **multiple** sets of accepting states $\mathcal{F} = \{F_1, \dots, F_n\}$. An execution is accepted if it visits *at least one state from each set* infinitely often.



Bridging Math and Software: The LGBA

4 A Tableau Construction

The Problem: Pure Automata Run “Blind”

- The automata defined so far have no input, and hence do not define any sequences.
- We need a way to read the protocol’s real events.

The Solution: Labeled States

We construct a **Labeled** Generalized Büchi Automaton (LGBA). Unlike common approaches that add labels to transitions, this algorithm labels the **states**. The labeling function is $\mathcal{L} : Q \rightarrow 2^{\mathcal{D}}$.



The Tableau Engine: Splitting Time

4 A Tableau Construction

The Core Strategy

- The algorithm builds a graph by decomposing formulas according to their Boolean structure.
- **Crucial Idea:** Separate what must be true **immediately** from what must be true **from the next state on**.

The Fundamental Identity

$$\mu \cup \psi \equiv \psi \vee (\mu \wedge X(\mu \cup \psi))$$

*Meaning: “ μ Until ψ ” is satisfied if either ψ is true **now**, OR (μ is true **now** AND the whole rule applies to the **next** state).*



The Node Data Structure (1/2)

4 A Tableau Construction

To implement the Depth-First Search (DFS) on-the-fly approach, each graph node contains specific fields.

- **Name:** A string representing the name of the node.
- **Incoming:** The edges represented by the names of nodes with an outgoing edge leading here.
 - *Note:* The special dummy name `init` is used to mark initial nodes.
- **Father:** The node from which the current one has been split.
 - *Note:* Used *only* for reasoning about correctness, not for actual construction.

Name: Node1
Father: Node0
Incoming: `init`
New: $\{\mu \cup \psi\}$
Old: $\emptyset$
Next: $\emptyset$



The Node Data Structure (2/2): The Engine

4 A Tableau Construction

These three fields handle the temporal logic processing over time:

- **New:** The “To-Do List”. Temporal properties that must hold in the current state but *have not yet been processed*.
- **Old:** The “Archive”. Properties already processed. A node is fully constructed when **New** becomes empty.
- **Next:** The “Legacy”. Temporal properties that must hold in all immediate *successors*.

Name: Node1
Father: Node0
Incoming: init
New: $\{\mu \cup \psi\}$
Old: $\emptyset$
Next: $\emptyset$

Global Memory

A global list `Nodes_Set` keeps track of nodes whose construction was completed, to detect loops and merge identical paths.



The Algorithm: Preparation & Simplifications

4 A Tableau Construction

Before building the graph, clean the formula:

- **Temporarily ignore 'X':** The Next operator is initially excluded to simplify the basic proof (added back later).
- **Remove 'F' and 'G':** Translated into 'U' (Until) and 'V' (Release) operators.

The Negation Trick

All negations ($\neg$) are pushed inside the formula until they precede only atomic propositions (creating literals).

Why? Using the dual operator 'V' allows pushing negations inward **without** causing an exponential blowup in the formula's size!



The Algorithm: Initialization & The Main Loop

4 A Tableau Construction

Step 1: The Initial Node

- $\text{Incoming} = \{\text{init}\}$
- $\text{New} = \{\varphi\}$ (The entire prepared formula)
- $\text{Old} = \emptyset, \text{Next} = \emptyset$

Step 2: Check the "New" field

- **If New is EMPTY:** The node is fully processed.
 - Check Nodes_Set for duplicates (same Old and Next).
 - *If duplicate:* Merge incoming edges, discard current node.
 - *If unique:* Add to Nodes_Set , create the successor node (moving Next into the successor's New).



Processing Formulas: Literals and AND

4 A Tableau Construction

If **New** is **NOT EMPTY**: Extract a formula η from **New**.

Case 1: η is a Literal (p or $\neg p$)

- **Contradiction Check:** If $\neg\eta$ is already in **Old**, the node contains a logical impossibility. **Discard the node immediately.**
- **Otherwise:** Add η to the **Old** archive.

Case 2: The AND Operator ($\mu \wedge \psi$)

- No splitting required.
- Both μ and ψ must be true, so add **both** to the **New** field of the current node.



Processing Formulas: Splitting Nodes

4 A Tableau Construction

When encountering branching logic, the node splits into two copies processed in **Depth-First Search (DFS)** order.

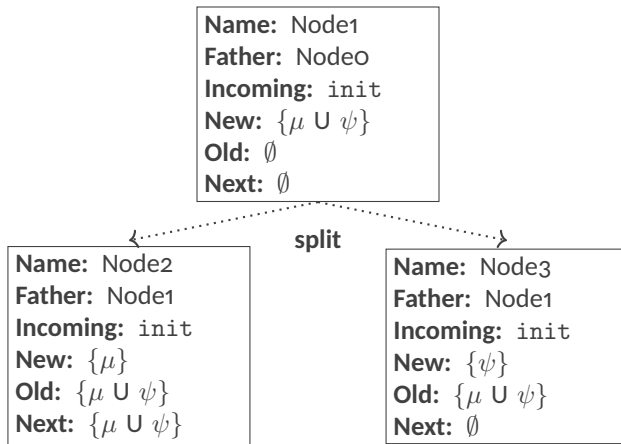
- **OR ($\mu \vee \psi$):**
 - Copy 1: Adds μ to New.
 - Copy 2: Adds ψ to New.
- **UNTIL ($\mu \mathbf{U} \psi$):**
 - Copy 1 (Delay): Adds μ to New, and $\mu \mathbf{U} \psi$ to Next.
 - Copy 2 (Solve now): Adds ψ to New.
- **RELEASE ($\mu \mathbf{V} \psi$):**
 - Copy 1 (Delay): Adds ψ to New, and $\mu \mathbf{V} \psi$ to Next.
 - Copy 2 (Solve now): Adds both μ and ψ to New.



Visualizing the Node: Splitting (UNTIL Operator)

4 A Tableau Construction

Example: Processing the Until Operator ($\mu \cup \psi$)





Visualizing the Node: Splitting (OR Operator)

4 A Tableau Construction

Example: Processing the OR Operator ($\mu \vee \psi$)

Name: Node1
Father: Node0
Incoming: init
New: $\{\mu \vee \psi\}$
Old: $\emptyset$
Next: $\emptyset$

split



Visualizing the Node: Splitting (OR Operator)

4 A Tableau Construction

Example: Processing the OR Operator ($\mu \vee \psi$)

Name: Node1
Father: Node0
Incoming: init
New: $\{\mu \vee \psi\}$
Old: $\emptyset$
Next: $\emptyset$

split

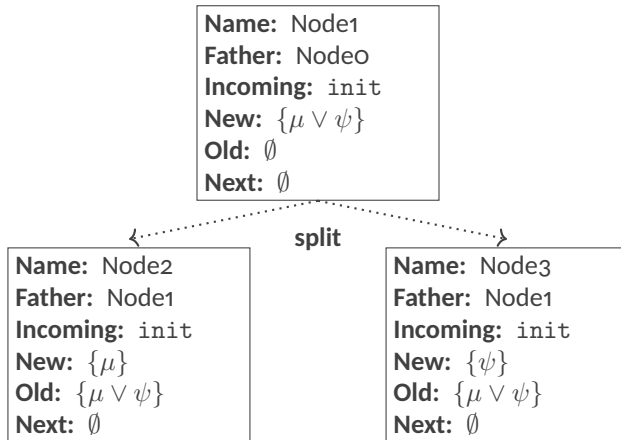
Name: Node2
Father: Node1
Incoming: init
New: $\{\mu\}$
Old: $\{\mu \vee \psi\}$
Next: $\emptyset$



Visualizing the Node: Splitting (OR Operator)

4 A Tableau Construction

Example: Processing the OR Operator ($\mu \vee \psi$)





Defining the Final Automaton

4 A Tableau Construction

The graph constructed by the algorithm is now mapped to a Labeled Generalized Büchi Automaton (LGBA).

Mapping the Topology:

- **States (Q):** The set of nodes returned by the algorithm (only nodes where the `New` field is empty).
- **Initial States (I):** Those nodes q that have the dummy label `init` in their `Incoming` field.
- **Transitions ($\rightarrow$):** A transition $p \rightarrow q$ exists exactly if the name of p is in `Incoming( $q$ )`.



Table of Contents

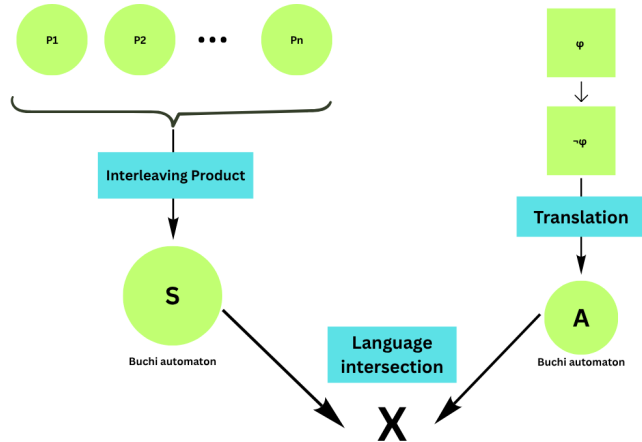
5 Example

- ▶ Abstract
- ▶ Introduction
- ▶ Preliminaries
- ▶ A Tableau Construction
 - The algorithm
 - Using the Automaton for Automatic Protocol Verification
- ▶ **Example**
- ▶ Nuances and summary
 - Improvements to the Basic Algorithm
- ▶ Experimental Results
- ▶ Conclusions
- ▶ Other stuff



How it all works together?

5 Example





LTL to Buchi: example of generation

5 Example



Table of Contents

6 Nuances and summary

- ▶ Abstract
- ▶ Introduction
- ▶ Preliminaries
- ▶ A Tableau Construction
 - The algorithm
 - Using the Automaton for Automatic Protocol Verification
- ▶ Example
- ▶ **Nuances and summary**
 - Improvements to the Basic Algorithm**
- ▶ Experimental Results
- ▶ Conclusions
- ▶ Other stuff



The Labels: Intersecting with Reality

6 Nuances and summary

To read the actual protocol's state, we must extract labels from the *Old* field of each fully expanded node.

Positive and Negative Constraints

Let $\mathcal{P}$ be the set of all atomic propositions. For each node q :

- $Pos(q) = Old(q) \cap \mathcal{P}$ (Propositions that **MUST** be true).
- $Neg(q) = \{\eta \mid \neg\eta \in Old(q) \wedge \eta \in \mathcal{P}\}$ (Propositions that **MUST** be false).

The Labeling Function $\mathcal{L}(q)$:

Contains any subset of atomic propositions that agrees with $Pos(q)$ and does not contradict $Neg(q)$. *Variables not mentioned in Old are simply ignored!*



Acceptance Conditions: Anti-Procrastination

6 Nuances and summary

The "Until" Problem

The algorithm allows a path to infinitely contain $\mu \text{ U } \psi$ (by pushing it to Next) without ψ ever actually occurring. We must forbid this!

Imposing Generalized Büchi Conditions ($\mathcal{F}$)

For **each** subformula of the type $\mu \text{ U } \psi$, we create one specific acceptance set $F \in \mathcal{F}$.

A node q belongs to F if:

- $\mu \text{ U } \psi \notin \text{Old}(q)$ (The promise is no longer active), **OR**
- $\psi \in \text{Old}(q)$ (The promise has been fulfilled today).

The automaton only accepts executions passing through all sets in $\mathcal{F}$ infinitely often, guaranteeing all "Until" promises are kept.



Improvements: The 'Next' Operator & On-Demand

6 Nuances and summary

Adding Nexttime Formulas (X)

- The initial restriction on the 'X' operator is lifted trivially.
- **Action:** If $\eta = X\mu$, simply add μ to the Next field of the current node. No splitting required!

Pure "On-The-Fly" Construction

We **do not** need to complete the property automaton before model-checking.

- Nodes are constructed *on-demand* while intersecting with the protocol.
- Only successors that match the *current protocol state* are generated.
- **Result:** Bugs can be found without ever generating the full property automaton, drastically saving time and memory!



Efficiency Improvements: Structural Tweaks

6 Nuances and summary

To transition from a mathematical proof to a fast software implementation, several redundancies are removed:

- **Removing the Father Field:** It is only needed for correctness proofs, not for finding bugs.
- **Smarter Splitting (Memory Saving):**
 - When a node splits (e.g., for an OR), we don't need to allocate memory for two entirely new nodes.
 - Instead, we update the current node in-place with the first option, and only create *one* new node for the second option.
- **Trivial Acceptances:** A formula like $\eta \text{ U } T$ (Until True) is logically equivalent to True. It does **not** generate a new acceptance set in $\mathcal{F}$.



Efficiency Improvements: Logic & Storage

6 Nuances and summary

Proactive Contradiction Detection

- Instead of waiting for atomic literals to clash, detect inconsistencies early using syntax.
- Before adding a formula μ , compute $\neg\mu$ and check if it already exists in the node. If so, abandon the node immediately.

Smart Storage in Old & Short-circuiting

- **Keep it clean:** Do not store every intermediate formula in Old. E.g., for $\mu_1 \wedge \mu_2$, storing the components is enough.
Fewer formulas = smaller nodes = higher chance to merge duplicate paths!
- **Short-circuiting:** If processing $\mu \cup \psi$ and ψ is *already* in $\text{New} \cup \text{Old}$, do not split the node. The goal is already achieved!



The Final Step: On-The-Fly Model Checking

6 Nuances and summary

Finding the Bugs:

1. Construct the property automaton for the **negation** of the desired property.
2. Intersect it with the protocol's automaton. Since the protocol has no accepting conditions ($\mathcal{F} = \emptyset$), the product inherits the property's accepting sets.

The Cycle Detection Trick

Standard DFS algorithms easily find cycles in *simple* Büchi automata, not generalized ones.

Solution: Add a counter i (modulo n , where $n = |\mathcal{F}|$) to the states. The counter increases when passing through the i -th acceptance set. A cycle containing all sets simply becomes a cycle resetting the counter!



Table of Contents

7 Experimental Results

- ▶ Abstract
- ▶ Introduction
- ▶ Preliminaries
- ▶ A Tableau Construction
 - The algorithm
 - Using the Automaton for Automatic Protocol Verification
- ▶ Example
- ▶ Nuances and summary
 - Improvements to the Basic Algorithm
- ▶ **Experimental Results**
- ▶ Conclusions
- ▶ Other stuff



Experimental Results: The Benchmark

7 Experimental Results

The Setup

- Both the classical "Global Construction" and the "New Construction" were implemented in Standard ML of New Jersey for a fair comparison.

Num.	Formula	Global Construction		New Construction		
		Nodes	Transitions	Nodes	Transitions	Accepts
1	$p_1 \cup p_2$	8	34	3	4	1
2	$p_1 \cup (p_2 \cup p_3)$	26	240	4	6	2
3	$\neg(p_1 \cup (p_2 \cup p_3))$	26	240	7	15	0
4	$GFp_1 \rightarrow GFp_2$	114	763	9	15	2
5	$Fp_1 \cup Gp_2$	56	337	8	15	2
6	$Gp_1 \cup p_2$	13	63	5	6	1
7	$\neg(FFp_1 \leftrightarrow Fp_1)$	-	-	22	41	2



Analyzing the Results: Key Takeaways

7 Experimental Results

Why these specific formulas?

- **Formula 4** ($GFp_1 \rightarrow GFp_2$): A standard "fairness" condition. The global method creates an enormous 114 nodes; the new algorithm needs only **9 nodes**.
- **Formulas 2 & 3 (The Negation Test)**: Formula 3 is the exact negation of Formula 2. The new algorithm handles negation seamlessly (from 4 to 7 nodes), avoiding the exponential blowup typical of older methods.
- **Formula 7 (Out of Memory)**: Checking if $(FFp_1 \leftrightarrow Fp_1)$ is a tautology. The global construction **crashed** due to insufficient memory (represented by "-"). The new construction solved it easily with just 22 nodes!



Table of Contents

8 Conclusions

- ▶ Abstract
- ▶ Introduction
- ▶ Preliminaries
- ▶ A Tableau Construction
 - The algorithm
 - Using the Automaton for Automatic Protocol Verification
- ▶ Example
- ▶ Nuances and summary
 - Improvements to the Basic Algorithm
- ▶ Experimental Results
- ▶ **Conclusions**
- ▶ Other stuff



Conclusions: Why This Matters

8 Conclusions

The Exponential Blowup Problem

- Traditional global construction suffers from memory and time blowup much faster.
- This is fatal because the final model-checking execution time is linearly proportional to the size of the constructed property automaton.

The "On-The-Fly" Advantage

In real-world model checking, the protocol's state space is already massive. Keeping the property automaton small is critical.

Because this algorithm operates strictly on-the-fly, **a violation can be found and reported before** the entire property automaton or protocol state space is fully constructed.



Table of Contents

9 Other stuff

- ▶ Abstract
- ▶ Introduction
- ▶ Preliminaries
- ▶ A Tableau Construction
 - The algorithm
 - Using the Automaton for Automatic Protocol Verification
- ▶ Example
- ▶ Nuances and summary
 - Improvements to the Basic Algorithm
- ▶ Experimental Results
- ▶ Conclusions
- ▶ Other stuff



The Formula and the Key Expansion Rule

9 Other stuff

We want to build a Büchi automaton for:

$$\varphi = p \cup q$$

Meaning: p holds at every step *until* q eventually holds.

Core expansion rule (used by SPIN's GPVW algorithm)

$$p \cup q \equiv q \vee (p \wedge X(p \cup q))$$

Either q holds **now**, or p holds **now** and the obligation $p \cup q$ is pushed to the **next step**.



Algorithm Overview — GPVW Node Structure

9 Other stuff

The algorithm builds a graph of **nodes**; each node eventually becomes an automaton state. Four fields per node:

Field	Contents	Role
Incoming	set of predecessors (or <code>init</code>)	who points here
Now	formulas yet to be processed	current obligations
Old	formulas already processed	constraint on state label
Next	formulas for the next node	successor obligations

Two key rules

1. $\text{Now} = \emptyset$: node = **automaton state**. A fresh successor node is created with $\text{Now} = \text{Next}$.
2. If a new node has the same (Old, Next) as an existing state s : **merge** — add the edge to s instead.



Step 1 — Create the Initial Node η_0

9 Other stuff

Place φ into Now of the starting node:

Incoming	Now	Old	Next
{init}	$\{p \cup q\}$	$\emptyset$	$\emptyset$

Action: pick $p \cup q$ from Now and apply the expansion rule:

$$p \cup q \longrightarrow \underbrace{q}_{\text{branch A}} \vee \underbrace{p \wedge X(p \cup q)}_{\text{branch B}}$$

$\Rightarrow$ Split η_0 into **two new nodes** η_1 (branch A) and η_2 (branch B).



Step 2 — Split: Nodes η_1 and η_2

9 Other stuff

Node η_1 (Branch A: q now)

Inc.	$\{\text{init}\}$
Now	$\{q\}$
Old	$\{p \cup q\}$
Next	$\emptyset$

q must hold *right now*.
No successor obligation.

Node η_2 (Branch B: p now, defer)

Inc.	$\{\text{init}\}$
Now	$\{p\}$
Old	$\{p \cup q\}$
Next	$\{p \cup q\}$

p must hold *now*.
 $p \cup q$ pushed to next step.

Both nodes inherit Incoming = $\{\text{init}\}$ and have $p \cup q$ moved from Now to Old.



Step 3 — Process η_1 : q is a Literal

9 Other stuff

Move q from Now to Old:

Incoming	Now	Old	Next
$\{\text{init}\}$	$\emptyset$	$\{p \cup q, q\}$	$\emptyset$

Now = $\emptyset \Rightarrow$ create automaton state s_1 :

State label (what must hold): q

Incoming edge from: init

Accepting? **Yes** — $p \cup q \in \text{Old}$ and $q \in \text{Old}$
(obligation fulfilled!)

Spawn a successor node η_3 with Now = Next = $\emptyset$.



Step 4 — Process η_3 : Empty Node $\rightarrow$ Sink State

9 Other stuff

Incoming	Now	Old	Next
$\{s_1\}$	$\emptyset$	$\emptyset$	$\emptyset$

Now = $\emptyset$ and no existing node matches (Old = $\emptyset$, Next = $\emptyset$).

$\Rightarrow$ **Create state** s_2 (no pending obligations anywhere):

State label: $\top$ (any valuation allowed)

Incoming edge from: s_1

Accepting? **Yes** — no pending Until-obligation

Spawn successor η_4 with Now = $\emptyset$. Its (Old, Next) are identical to $s_2 \Rightarrow$ **merge**: add s_2 to s_2 's incoming.

s_2 gets a self-loop on $\top$



Step 5 — Process η_2 : p is a Literal

9 Other stuff

Move p from Now to Old:

Incoming	Now	Old	Next
$\{\text{init}\}$	$\emptyset$	$\{p \cup q, p\}$	$\{p \cup q\}$

Now = $\emptyset \Rightarrow$ create automaton state s_3 :

State label (what must hold): $p \wedge \neg q$ (p is in Old; q is not)

Incoming edge from: init

Accepting? **No** — $p \cup q \in \text{Old}$ but $q \notin \text{Old}$

Spawn successor η_5 with Now = $\{p \cup q\}$ (from Next).



Step 6 — Successor η_5 of s_3 Loops Back

9 Other stuff

Incoming	Now	Old	Next
$\{s_3\}$	$\{p \cup q\}$	$\emptyset$	$\emptyset$

Action: expand $p \cup q$ again — same split as Step 1!

Branch A (q holds now)

Old = $\{p \cup q, q\}$, Next = $\emptyset$

Matches s_1 ! **Merge:**

add s_3 to s_1 's Incoming.

$$s_3 \xrightarrow{q} s_1$$

Branch B (p holds, defer)

Old = $\{p \cup q, p\}$, Next = $\{p \cup q\}$

Matches s_3 ! **Merge:**

add s_3 to s_3 's Incoming.

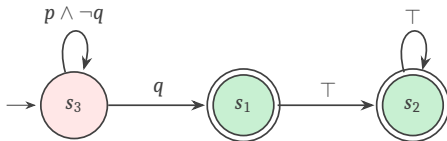
$$s_3 \xrightarrow{p \wedge \neg q} s_3$$

No new states created. **Construction complete!**



Final Büchi Automaton for $p \text{ U } q$

9 Other stuff



Valid run example:

$s_3, s_3, \dots, s_3, s_1, s_2, s_2, \dots$

($p \wedge \neg q$ holds, then q holds, then anything forever)

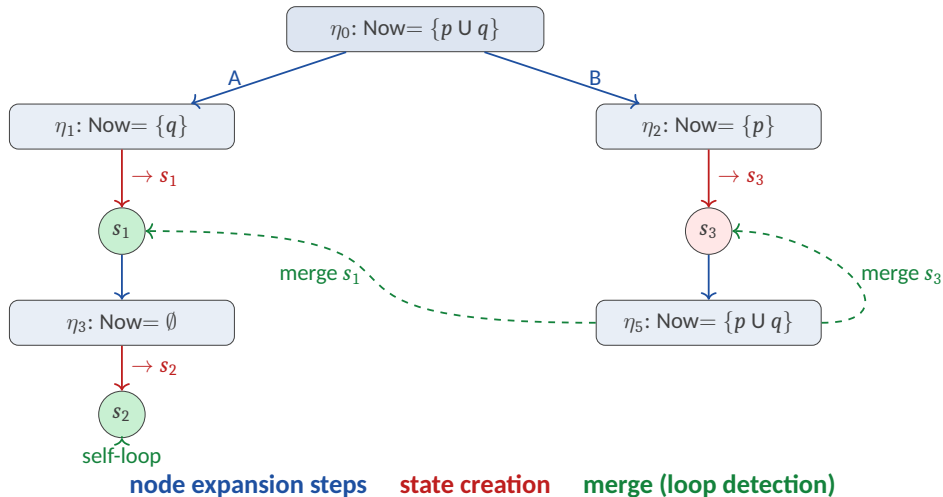
Visits s_1 or s_2 (both accepting) infinitely often $\Rightarrow$ accepted. Looping forever in s_3 (non-accepting) $\Rightarrow$ rejected.

State	Label	Acc.
s_3 (init)	$p \wedge \neg q$	No
s_1	q	Yes
s_2	$\top$	Yes



Summary of the Construction

9 Other stuff



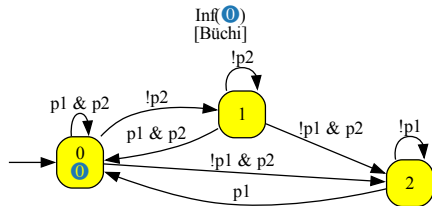
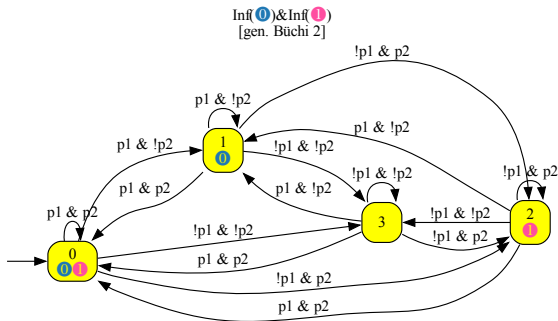


In Practice: From Generalized to Simple Automaton

9 Other stuff

Generalized (LGBA) –S 'GFa & GFb'

Simple Büchi (BA) –B 'GFa & GFb'



- Uses **multiple** acceptance sets

- Uses a **single** acceptance set.
- The automaton is flattened using the "counter trick" to allow standard DFS cycle detection.



Syntactic Sugar: Derived Operators

9 Other stuff

To keep formulas compact, standard abbreviations are introduced:

- **True/False:** $T \equiv p \vee \neg p$, $F \equiv \neg T$.
- **Eventually (F):** $F\varphi \equiv T \cup \varphi$
Meaning: φ will become true at some point in the future.
- **Globally (G):** $G\varphi \equiv \neg F\neg\varphi$
Meaning: φ is true now and will stay true forever.

The “Release” Operator (V)

Defined as the dual of Until: $\varphi V \psi \equiv \neg(\neg\varphi U \neg\psi)$.

Crucial for pushing negations inside the formula without causing exponential blowup!